

ReceiptIQ - Server Guide

How the project runs on the server - English

1. The app does not need to be started

ReceiptIQ runs on the server under **systemd**. It starts on boot, restarts itself on crash, and keeps running whether or not anyone is logged in.

No command is needed to keep it alive. Closing SSH, shutting the laptop, or disconnecting from OpenVPN has no effect on it - the application lives on the server, not on the workstation.

2. What runs on the server

Service	Role
receiptiq	Django application via gunicorn, on port 8001
receiptiq-ngrok	Public route that works without the VPN
receiptiq-tunnel	Cloudflare backup route
postgresql	Database (receiptiq_db)
ollama	Local AI models, bound to localhost:11434 only

3. Which address to use

OpenVPN	Address
Connected	http://10.0.2.13:8001
Disconnected	https://unaroused-uncontributively-kalyn.ngrok-free.dev
Disconnected	https://earth-prediction-hostels-optimization.trycloudflare.com (backup)
Disconnected	http://10.0.2.13:8001 - will NOT work

Why: 10.0.2.13 is an RFC1918 private address. It is only meaningful inside the VPN, and is not routable on the public internet. When the VPN drops, the route disappears - the application is still running, only the path to it has to change.

4. Where configuration lives

The port and the public URL are set in the project's own **.env** file:

```
~/receiptiq/.env

APP_PORT=8001
GUNICORN_WORKERS=3
GUNICORN_TIMEOUT=600
NGROK_URL=https://unaroused-uncontributively-kalyn.ngrok-free.dev
```

Each service runs a small wrapper script that sources **.env** and passes the values on. No port or URL is hardcoded in the unit files:

```
~/receiptiq/deploy/run-app.sh -> gunicorn, bound to APP_PORT
~/receiptiq/deploy/run-ngrok.sh -> ngrok, serving NGROK_URL
~/receiptiq/deploy/run-cloudflared.sh -> Cloudflare tunnel
```

```
/etc/systemd/system/receiptiq.service
ExecStart=/home/siresoft/receiptiq/deploy/run-app.sh
```

To change the port, edit `.env` and restart the services. All three pick up the new value.

```
sudo systemctl restart receiptiq receiptiq-ngrok receiptiq-tunnel
```

The wrappers exist because systemd cannot read `.env` itself: it loads `EnvironmentFile` as PID 1, and SELinux (Enforcing on this host) denies `init_t` access to `user_home_t`. The wrapper runs as `siresoft`, who owns the file. Note also that scripts under `/home` need `'chcon -t bin_t'` or systemd refuses to execute them (status=203/EXEC).

Note: Django's `settings.py` never holds the port. Django has no knowledge of the port it is served on - gunicorn owns that.

5. Day-to-day commands

Connect:

```
ssh siresoft@10.0.2.13
cd ~/receiptiq
```

Health check - this covers most situations:

```
systemctl is-active receiptiq receiptiq-ngrok receiptiq-tunnel
```

Three lines, all of which should read 'active'.

Confirm the app is answering:

```
curl -I http://10.0.2.13:8001/
```

HTTP 302 or 200 means it is healthy.

After changing code or `.env`:

```
sudo systemctl restart receiptiq
```

Logs:

```
journalctl -u receiptiq -n 50 --no-pager # last 50 lines
journalctl -u receiptiq -f # live (Ctrl+C to quit)
journalctl -u receiptiq -p err -n 30 --no-pager # errors only
```

6. Django tasks

```
cd ~/receiptiq
source venv/bin/activate

python manage.py migrate
python manage.py createsuperuser
python manage.py collectstatic --noinput

deactivate
sudo systemctl restart receiptiq
```

To test code changes, always use a spare port - 8001 belongs to the live app:

```
python manage.py runserver 0.0.0.0:8002
```

7. Commands to treat with care

Command	Effect
<code>sudo systemctl stop receiptiq</code>	Takes the application down
<code>sudo systemctl stop receiptiq-ngrok</code>	Removes the VPN-free route
<code>sudo systemctl disable receiptiq</code>	Stops it starting at boot
<code>rm</code>	Deletes permanently
<code>cat > file</code>	Truncates the file immediately

If the screen freezes showing ':' or '(END)', press **q**. If the shell is stuck at a '>' prompt, press **Ctrl+C**.

8. Outstanding work

The public URLs above are free tunnel addresses. They are fine for testing and demos, but are not suitable as the company's production address.

A stable hostname such as **https://receipts.siresoft.com** requires one of the following. Both are network/DNS tasks rather than application changes:

1. Move one siresoft domain onto a free Cloudflare account. Cloudflare Tunnel then provides the hostname and an HTTPS certificate at no cost. Roughly 10 minutes of work.
2. Or add a port-forward rule on the gateway (10.0.2.1): external TCP 8001 -> 10.0.2.13:8001, together with a DNS record and nginx + Let's Encrypt on the server.

ngrok was also evaluated. It charges for custom subdomains, so it is not the right route for a named address.

siresoft.com currently runs on its own nameservers (ns1/ns2.siresoft.com), so this needs someone with DNS access.